

Requirements Capture and Analysis in ASSERT™

Andrew Crapo, Abha Moitra
GE Global Research
Niskayuna, NY 12309, USA
crapo@research.ge.com

Craig McMillan, Dan Russell
GE Aviation Systems
Grand Rapids, MI 49512, USA

Abstract—Capturing high-level requirements in a human readable but formal representation suitable for analysis is an important goal for GE. To that end we have augmented an existing controlled-English modeling language with a new controlled-English requirements capture language to create the Requirements Capture frontend of the ASSERT™ tool suite¹. Requirements captured in ASSERT™ can be analyzed for a number of possible shortcomings, both individually and collectively. Once a set of requirements has reached a satisfactory level of completeness, consistency, etc., it can then be further used to generate test cases and test procedures. This paper will focus on the requirements capture and analysis functions of ASSERT™ and will illustrate its capabilities with a sample problem previously used as a challenge problem for requirements specification.

Index Terms—Model, requirements capture, formal methods.

I. INTRODUCTION

The conference theme, “Desperately Seeking Less: The Role of Simplicity and Complementarity in Requirements”, provides an interesting launch pad for the topic of this paper. While it is intuitive that “seeking less” can lead to “simplicity”, one assumes that a certain level of specificity and disambiguation must be preserved. Otherwise, as Feynman [1] pointed out, we can simply write “ $U=0$ ” as the complete expression of any set of requirements.

The quest for simplicity is perhaps more accurately a quest for understandability. One might argue that any expression which is clearly understood by an observer is, to that observer, relatively simple. For example, Sean Carroll [2] has suggested that the laws of physics governing everyday life can be summarized in a single equation as in Figure 1. For someone with his background, this captures in an understandable fashion a great deal of information about physics.

What kind of representation would be easily understood by a requirements engineer? We will suggest in this paper that at least one answer to that question is requirements written in a language as close as possible to English, using common names for the concepts of the domain, and yet formal enough that requirements expressed in the language may be analyzed using formal methods.

II. RELATED WORK

There are other tools that capture formal requirements and analyze them [3, 4, 5]. One tool in particular is SpeAR – Specification and Analysis of Requirements [6, 7], which is an open

source tool and language that allows for capturing and analyzing requirements. SpeAR uses semantics of the Lustre synchronous dataflow programming language and Past Linear Temporal Logic (Past LTL) to express temporal relationships. SpeAR supports requirements analysis, real-time type checking, dimensional analysis of unit computations, along with other checks to provide feedback to users assisting with elaboration of well-formed requirements. Some features of SpeAR are comparable to our tool, though we believe that our tool differs in the following ways: 1) we rely on ontology-based technologies, which allow for automated reasoning about concepts and relationships in the domain relevant to the system and to the requirements that are being refined and analyzed, and 2) we generate tests based on requirements all before a single line of executable code is written. ASSERT™ packages all these capabilities in a tool suite.

III. A GROUNDING EXAMPLE

To ground our discussion and demonstrate the methodology used, we chose an historical challenge problem as an illustrative example: the steam-boiler control specification problem. The steam-boiler problem was initially posed by Bauer [8] and was the subject of a Dagstuhl workshop with twenty-two solutions [9]. The challenge is to create the specification of a program to control the level of water in a steam-boiler. The physical system consists of the following.

1. A steam-boiler
2. A device to measure the quantity of water in the steam-boiler
3. Pumps to cause water to flow into the steam-boiler (the original had 4 but we will use at least 1)
4. A pump controller (the original had 4, one for each of the pumps)
5. A device to measure the quantity of steam flowing out of the steam-boiler
6. A messaging system connecting the physical elements

$$W = \int_{k < \Lambda} [Dg][DA][D\psi][D\Phi] \exp \left\{ i \int d^4x \sqrt{-g} \left[\frac{m_p^2}{2} R - \frac{1}{4} F_{\mu\nu}^a F^{a\mu\nu} + i \bar{\psi}^i \gamma^\mu D_\mu \psi^i + (\bar{\psi}_L^\dagger V_{ij} \Phi \psi_R^j + \text{h.c.}) - |D_\mu \Phi|^2 - V(\Phi) \right] \right\}$$

Figure 1. Sean Carroll's 'Big Picture' Equation.

¹ ASSERT™ stands for Analysis of Semantic Specifications and Efficient generation of Requirements based Tests.

to the control program (we will exclude this from our example for simplicity)

The original challenge problem had two different minimum and maximum water quantities, one for normal operation and another to signal a hazardous state. For simplicity, we only model the second limits. Similarly, there are several modes of operation: initialization, normal, degraded, rescue, and emergency stop. For simplicity, we dropped initialization.

IV. THE VALUE OF A DOMAIN MODEL

Preciseness suggests carefully unambiguous. Whether written or oral, preciseness in communication is more easily obtained by using words which have, at least in context, a single clear meaning shared by practitioners in the domain. This provides the impetus for domain “jargon”. By domain model, we mean a formal representation of the concepts (names) that are used in the domain and what those concepts mean. By formal we mean that the model has meaning “understandable” by machines as well as humans. Ontologies provide a mechanism for creating such models.

Most if not all ontology languages are based on set theory and some fragment of logic. Set theory is very much like the generalizations that humans naturally and informally use in constructing natural language. It is hypothesized to be aligned with our mental models [10, 11, 12]. Because it is formalized, set theory provides a theoretical underpinning for defining classes or kinds of things and partially ordering them into class hierarchies. This in turn enables inheritance, which makes it possible to be parsimonious and describe the characteristics of a class in a single location in a model, with all subclasses inheriting those characteristics. Set theory also underpins the definition of members of a class, referred to as instances or individuals.

Properties are another type of generalization, this time about the kinds of characteristics things have and the kinds of relationships that things can have with other things. Like classes, properties can be partially ordered into hierarchies. The final core building blocks for ontologies are logical axioms that impose constraints on what kinds of relationships can or must exist. In many ontology languages, these axioms are called restrictions and are defined in terms of a class and a property. Common kinds of axioms include a restriction that members of a class must have property values from another class, e.g., steam-boiler systems must contain pumps, a restriction that all the values of a property for members of a class must come from a particular class, e.g., all condition indicators of a steam-boiler system must be of type state, a restriction that a property of all members of a class must have a particular value, e.g., the fluid in all steam-boilers must be H_2O , and a variety of forms of cardinality restrictions, e.g., a steam-boiler has exactly 1 level, a steam-boiler must have at least 2 pipe connections, a steam-boiler system has exactly 1 component of type steam-boiler and at least 1 component of type pump, etc.

These basic ontology building blocks allow a domain model to be expressed in a way that gives formalism to the concepts—the artifacts, relationships, processes, etc.—of the domain. Once these domain concepts are defined, they may then

be used to express requirements. The domain model becomes the domain-specific language in which the requirements are written. This makes the requirements understandable to practitioners in the domain and the requirements therefore feel much simpler. They are expressed in terms compatible with subject matter experts’ mental models and compatible with the literature of the domain. The focus is shifted away from the formalism itself and towards the domain.

V. LANGUAGE CONSIDERATIONS

While there are a variety of languages available for building domain models, we use the Web Ontology Language (OWL) [13]. OWL is a recommendation of the World Wide Web Consortium (W3C) [14]. OWL is widely-used and is sufficiently expressive for our needs. OWL combines set theory with a decidable fragment of first order logic (FOL) and belongs to a class of languages called Decision Logics (DL). In OWL, restrictions on the properties of a class are called property restrictions, and these include all those mentioned in the previous section. There are different flavors of OWL with differing levels of expressivity and computational complexity and supporting differing sets of property restrictions. OWL has the distinct advantage of having a model theory and a proof theory. This means that, given a statement in a model, one can determine definitively whether the statement is true or false and that there is a specification of how to do so. In contrast, object-oriented languages such as Java, and even the UML modeling paradigm, have no model theory and proof theory so there is no definitive ground truth about what can correctly be inferred from a model in these representations.

OWL is not a natural language and none of its various serializations approach natural language in terms of human readability. This condition formed the motivation to build a controlled-English language called the Semantic Application Design Language (SADL), which is Open Source under the Eclipse Public License [15,16]. SADL can express all the constructs of OWL Version 1 and qualified cardinality from OWL Version 2.

SADL also supports rules as a means of capturing additional domain knowledge. A rule is closely aligned with implication in FOL with a condition side and a conclusion side. Rules are a part of the W3C technology stack. A rule language must be compatible with the ontology language and reasoners must take an integrated approach to the implications of logical inference and the implications of rule-based inference.

In addition, SADL supports the definition of equations, both user-defined equations and external equations. Both kinds of equations are specified in terms of their signature—the number, type, and order of inputs and outputs. User-defined equations include a specification of the equation body, an expression that specifies how the output is generated from the input. External equations include a uniform resource identifier (URI) to provide external identity. (URIs are the means of establishing identity in the Web.)

Finally, SADL provides a number of constructs to help test, debug, and maintain domain models. All valid SADL models are saved both in SADL syntax and in a user-specified standard

```

1 uri "http://sadl.org/BasicConcepts.sadl" alias basic version "1.0".
2
3 System is a class.
4 Subsystem is a type of System.
5 component describes System with values of type Subsystem.
6
7 Equipment is a type of Subsystem
8   described by state with a single value of type State.
9 State is a class, must be one of {Working, Failed}.
10
11 controls describes Equipment with values of type Equipment.

```

Figure 2. Basic Concepts Model.

serialization of OWL. If rules are present they are saved in a user-specified target rule language. Equations are likewise saved in a user-specified target representation, e.g., Prolog.

Besides being an ontology language, SADL is also an Xtext-based integrated development environment (IDE) [17]. This environment provides semantic coloring of different parts of speech in models, hyperlinking of concepts to their definitions and usage, integration with source code control systems via Eclipse plug-ins, graphical visualization of models, and other functionality useful for creating and maintaining models over their lifetime.

OWL, and therefore SADL because it maps to OWL, is a graph-theoretic language. This means that all constructs of a model can be represented as a set of statements, each of which has three parts: a subject, a predicate (or property), and an object (or value). These statements are called triples and coincide exactly with the mathematical definition of a directed graph: node $\rightarrow$ edge $\rightarrow$ node. The result is that domain models in SADL (OWL) can be quite easily rendered as visual graphs. Graphing capability is an integrated part of the SADL IDE.

With this introduction, we are ready to create the domain model for our steam-boiler problem. Our first model will be of some very basic concepts, shown in Figure 2 in SADL syntax. Line 1 in Figure 2 is the name of the model. In the Semantic Web, of which OWL is a part, identity is by URI. In this case the URI of the model (<http://sadl.org/BasicConcepts.sadl>) identifies the namespace of which the concepts defined in this model are a part. For example, this model defines a class whose name is *System* and whose identity is the URI <http://sadl.org/BasicConcepts.sadl#System>. The alias provides a prefix for qualified names, which are more human-readable. For example, instead of the complete URI we could use the qualified name *basic:System*. Using this and not just the name would be necessary if *System* were defined in more than one namespace used in the modeling effort.

The *basic* model further states that *Subsystem* is a class and a subclass (subset) of *System*. From the underlying set theory, this means, by definition, that every instance of a *Subsystem* is an instance of a *System*. The model also defines a property, *component*, that creates the aggregation of parts (*Subsystems*) to make wholes (*Systems*). But since every instance of a *Subsystem* is a *System*, an instance of a *Subsystem* can, in turn, have parts.

This model also says that the class *Equipment*, a subclass of *Subsystem*, is described by (is in the domain of) the property *state* and that the possible values of this property come from the class *State*, which has two instances, *Working* and *Failed*.

```

1 uri "http://sadl.org/SteamBoiler.sadl" alias boiler version "1.0".
2 import "http://sadl.org/BasicConcepts.sadl".
3
4 SteamBoilerSystem is a type of System
5   described by mode with a single value of type SteamBoilerSystemMode.
6 SteamBoilerSystemMode is a class, must be one of {Normal, Degraded, Rescue, Stopped}.
7 SteamBoiler is a type of Equipment
8   described by ^level with a single value of type decimal,
9   described by normalWaterLevel with a single value of type decimal
10  described by operationalWaterLevel with a single value of type decimal.
11 Pump is a type of Equipment.
12 PumpController is a type of Equipment.
13 component of SteamBoilerSystem has exactly 1 value of type SteamBoiler.
14 component of SteamBoilerSystem has at least 1 value of type Pump.
15 component of SteamBoilerSystem has at least 1 value of type PumpController.
16 controls of Pump has exactly 1 value of type PumpController.

```

Figure 3. Steam Boiler Model.

In other words, *State* is the range of *state*. It is convention but not necessary to start class names with an upper-case letter and property names with a lower-case letter².

With these few very basic concepts in place, consider next how to build upon them to create the model of a steam-boiler system shown in Figure 3. First note the import statement in line 2. This makes all the concepts defined in the *basic* model (and any of its imports were there any) available in this model. Note also that *System* is subclassed with *SteamBoilerSystem*, which can have a *mode* of *Normal*, *Degraded*, *Rescue*, or *Stopped*. *SteamBoiler* is defined as a subclass of *Equipment*, differentiated by being in the domain of three properties taking values of type *xsd:decimal*, e.g., *level*, *normalWaterLevel* and *operationalWaterLevel*. Note that the caret (^) in front of “level” is necessary because “level” is a keyword in the SADL grammar. By escaping it, we can also use “level” as a domain concept.

Lines 13 through 15 add qualified cardinality restrictions to the *SteamBoilerSystem* class, requiring that such a *System* have exactly one *component* of type *SteamBoiler*, at least one *component* of type *Pump*, and at least one component of type *PumpController*. For each *Pump*, there is a *PumpController* that “controls” it. These axioms significantly refine our model of a *SteamBoilerSystem*.

VI. THE SADL REQUIREMENTS LANGUAGE (SRL)

SADL (OWL) provides a language for creating a domain model, which then serves as a domain-specific language for talking about requirements, but SADL is not itself a requirements language. To provide the additional expressivity necessary to capture requirements we have created an extension of SADL called the SADL Requirements Language (SRL) [18]. SRL is very similar to the SADL rule language in some ways but uses some specialized keywords and constructs to make SRL requirements more similar to natural-language requirements. Like SADL, SRL comes with its own IDE with semantic coloring, linking to SADL model concepts, statement completion assistance, and graphing capability. SRL also has significant similarity to the templates used in EARS [19].

Like a rule, a requirement is expressed in terms of graph patterns to be matched in the domain model. SRL does not re-

² The existence of the class and the property with names differing only by capitalization is common and a reflection of the flexibility (and vagueness) of natural language, and related to the concept of “role”, which we will not go into in this paper.

quire that graph patterns be completely specified when omissions are not ambiguous, thereby allowing a level of parsimony demanded by requirements engineers. For example, one might have the following expression.

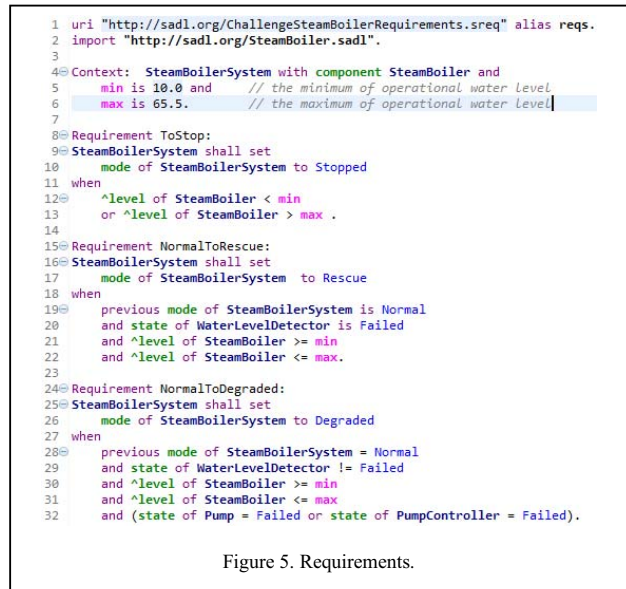
normalWaterLevel of component of SteamBoilerSystem

This kind of a statement is referred to as a “property chain”. Since only *SteamBoiler* is in the domain of *normalWaterLevel*, and *SteamBoiler* is in the range of *component*, and *SteamBoilerSystem* is in the domain of *component*, this statement is valid from a type-consistency perspective. However, requirements engineers want to be able to just say *normalWaterLevel* and let the rest be implied. SRL permits this if *normalWaterLevel* is unambiguous, meaning that because of property definition or restrictions, it is clear what the complete graph pattern would be. But suppose that the domain of *normalWaterLevel* were a union of multiple classes and that it made a difference which property chain was used. SRL allows an optional “Context” to be specified in a requirements model. If our set of requirements included other systems besides *SteamBoilerSystem*, the context might be:

Context: **SteamBoilerSystem**
with **component SteamBoiler**.

This serves to make unambiguous which part of the ontology to use when interpreting the requirements by expanding incomplete property chains. To help requirement writers understand the portion of the domain model being specified in a context, SRL provides context graphing capability. A context graph repeats nodes and subtrees so that one can easily trace all of the nodes and edges between a root concept and the leaves of the ontology. For example, Figure 4 shows a graph of the context in the previous paragraph.

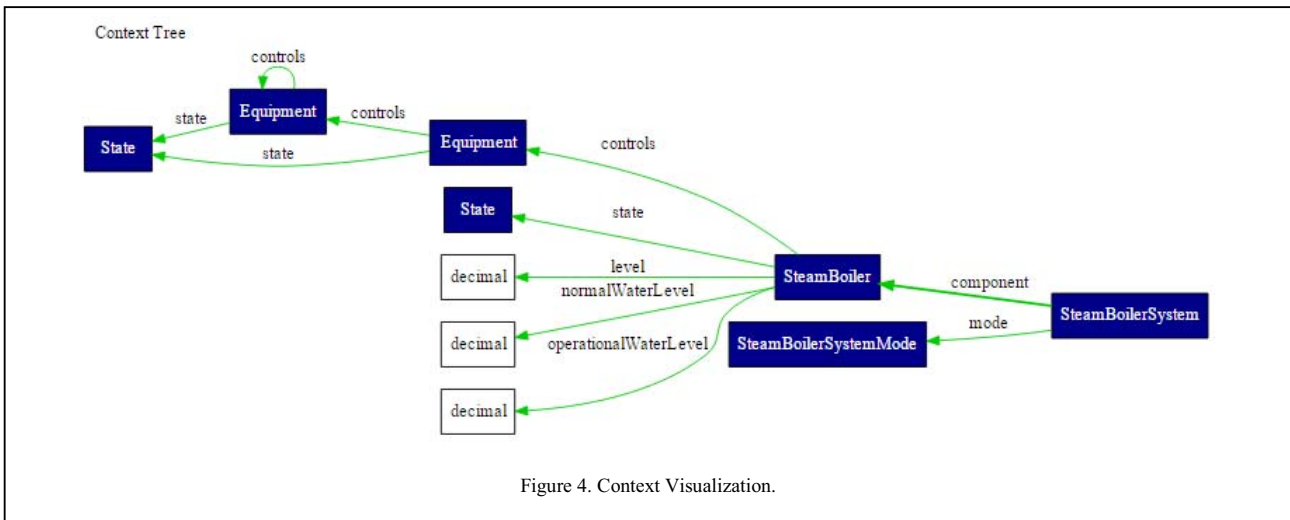
SRL supports various kinds of requirements. Figure 5 shows the first few requirements for the steam-boiler problem and the complete set is listed in Appendix A. The requirement *ToStop* says that the *SteamBoilerSystem* shall set its *mode* to



Stopped when the *level* of the *SteamBoiler* is below the minimum operating level or above the maximum operating level.

The second requirement, *NormalToRescue*, illustrates SRL’s implicit modeling of time. Since SRL is geared toward high-level requirements modeling, we avoid using a detailed time model by making some aspects of time implicit. The *previous* keyword used in front of a property (*mode* in this case) in a requirement condition is based on a discrete time model with an unspecified “time slice”. The requirement will set the current value of *mode* based on the value of *mode* at the previous time slice (*previous mode*) and other conditions.

In our requirement modeling we differentiate between ontology properties whose value is being set in the “shall” part of the requirement and those providing the value to be used or being tested in the conditional “when” part. The former we call “controlled properties”, the latter “monitored properties”. Note that in the case of the requirement *NormalToRescue*, *mode* is both monitored and controlled. This is only possible if the monitored usage is preceded by *previous*, and *previous* can



only appear with monitored properties.

Note that requirements can and often are expressed in terms of classes rather than instances. In the requirement *ToStop* in Figure 5, only *Stopped* is a specific instance. This requirement might be expressed as “Any instance of a steam-boiler system shall set the mode of the system to *Stopped* when the water level of the instance of steam-boiler which is a part of the system is outside the operational range.” When requirements are expressed in the generalities of classes rather than instances, some requirement analysis engines (see later section on analysis) may require “grounding” to instances of those classes specified in the ontology or to hypothesized instances of those classes.

Besides the kinds of requirements illustrated by the steam-boiler problem, SRL supports the following.

- Time-dependent conditions, with or without a specified interval:
 1. *<property> was <value>*
 2. *<property> was <value> in the past <n> <time-unit>*
 3. *<property> has been <value>*
 4. *<property> has been <value> for the past <n> <time-unit>*

The first condition is that some property was some value at an unspecified time in the past. The second condition adds a specific time interval ending now during which the condition was met. The third condition is that the property has always had the value. The fourth states that the property had the value during the entire specified interval ending now. Note that any of these time-dependent conditions can be negated, e.g., *<property> has not been <value>*.

- Events—an event is something that happens at a point in time. In our implicit model of time, *Event* is defined in an implicit model automatically included in every ontology. If a property has a range which is a subclass of *Event*, then a condition using that property will be considered a temporal requirement and the property may be followed by “in the past <n> <time-unit>”, meaning that the event must have occurred in the specified time-interval ending now. Without this explicit time interval, the event must have occurred in the single time slice ending now.
- Lists—a list is items in a specified order. Lists have been found to be important to requirements specification. The SRL grammar supports several list operations including insertion, removal, retrieval, replacement, and construction. There are also expressions to find the length of a list, to find the index of an element in a list, to test if a list contains an element, and to determine if an element is unique in a list.
- Tabular—a tabular requirement allows the monitored and controlled property values being tested or set to be placed in a table for a more compact and easily understood representation. A tabular requirement is equivalent to a set of requirements, one for each row in the table.

- Decomposition—sometimes a requirement is not completely specified at the current level, e.g., a high-level requirement. Decomposition allows an incomplete specification with some detail being provided as information to be used in a subsequent, lower-level specification.

SRL supports user-defined variables for increased parsimony and readability of requirements. A variable is defined with an equivalent expression and, once defined, can be used in other expressions, including variable definitions. In a pre-processing step, variables are replaced with their defining expression before the requirement is submitted to translation.

SRL variables can be defined in the context. Variable definition is a second purpose for specifying a context. A context variable can be used in any requirement within the requirement model. The context shown in Fig 5 defines the variables *min* and *max*. SRL variables can also be defined in the “where” clause of a requirement. This optional clause is like the “where” sometimes used in equation definitions in text books. The equation itself is expressed in a simpler, more readable form and is followed by a “where” part containing the definition of one or more intermediate variables used in the main equation.

SRL supports assumptions, although our example problem does not use them. An assumption is an additional constraint, possibly with conditions on the inputs, that holds for all requirements in the model (the SRL file). SRL also supports the concept of an assertion, a statement about what may be assumed to hold for the outputs of the model.

SRL includes a small language for specifying additional information necessary for test case generation. This grammar is called the “verify” grammar and will not be addressed in this paper.

Just as SADL models are saved both as text files which may be parsed by the Xtext-based SADL-IDE and as OWL files, SRL requirement models are saved both as text files which may be parsed by the Xtext-based SRL-IDE and as Prolog files. The Prolog output is in a form which we call “direct write Prolog”. It is a faithful representation of the requirements, assumptions, and assertions with variables eliminated.

VII. REQUIREMENTS ANALYSIS

Once a set of requirements have been captured, we want to analyze them for conflicts, completeness, independence etc. There are a number of tools available for such analyses and many of them require tool-specific representation of the requirements for input into the analysis. Since SRL is not a format known to any analysis tools, the SRL representation must be converted to the target representation. As we can envision different analysis tools for different types of analysis, we decided to do the translation of SRL into the target representation in 2 stages. The first stage converts requirements into a first-order-logic if-then form by eliminating context and expanding out incomplete graph patterns. Stage 1 translation is independent of the target analysis tool and its representation. Stage 1 translation is modular and each requirement can be translated independently of other requirements. Stage 2 translation converts this FOL form into the target representation.

Both Stage 1 and Stage 2 translation is accomplished using Prolog. Prolog is a declarative language well suited to recognizing complex patterns in the direct write, including incomplete property chains, and converting to the FOL form. This expansion and conversion is dependent upon the ontology, with its concept definitions and property restrictions. SWI-Prolog is well suited to the task as it is able to read and use OWL models [20].

A. Requirements Analysis Tool

The requirements analysis tool we are using is ACL2s [21]. ACL2s requires that all the requirements be grouped by the controlled variable and the target representation is Lisp. ACL2s runs a number of checks including contingency, conflict, completeness, independence and surjectivity [22].

B. Stage 2 Translation to ACL2s

The Stage 2 translation takes as input the FOL form output of Stage 1 translation and generates the ACL2s Lisp representation. Some of the important steps in this process are as follows

- Clausification: converts requirement expressions into normal form
- (Partial) Grounding: replaces classes in requirement expressions with their instances in all possible ways
- Simplification: eliminates tautologies, drops trivially true requirement instances, etc.
- ACL2s Expression Generation: converts resulting requirement instance expressions from first-order representation into ACL2s representation
- Implicit Requirement Generation: generates additional (implied) requirements
- Lisp File Generation: groups requirements by controlled variables
- Post Processing: performs inference over requirements to determine undefined ranges of variables in ACL2s representation

Stage 2 employs OWL and other reasoning paradigms to perform the above steps.

C. Analysis Results

Figure 6 shows the results of analyzing the requirements for the steam-boiler model. These results indicate that all the requirements passed all the checks performed.

To show the power of the framework we have developed, we can alter the requirements and see how the results are impacted.

- Experiment 1: If we comment out the line “and state of WaterLevelDetector != Failed” in requirement *NormalToDegraded* then we see a conflict pair identified by the analysis along with the details of the issue, shown in Fig 7.
- Experiment 2: Changing “>= min” to “> min” in the requirement *NormalToRescue* gives a completeness error as shown in Figure 8.

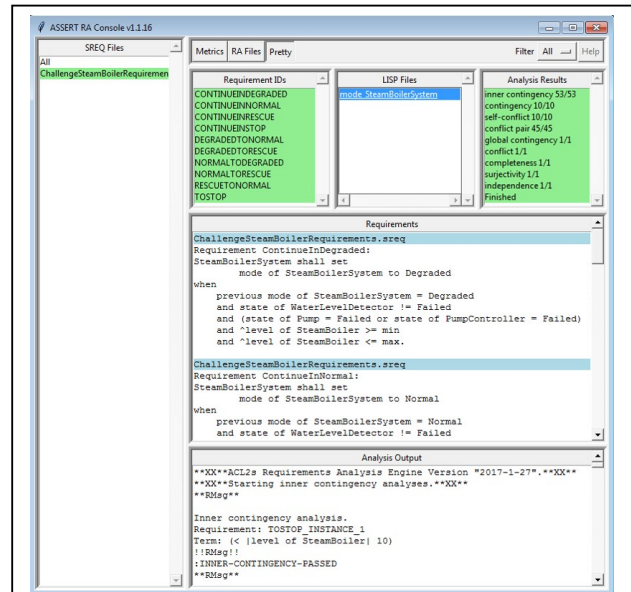


Figure 6. Requirement Analysis Results.

VIII. THE ASSERT™ TOOL

The requirements capture and analysis capability discussed in previous sections of this paper are part of the ASSERT™ platform developed at GE. The ASSERT™ platform does more than capture and analyze requirements. It also performs automated test case and test procedure generation. A simplified architecture of ASSERT™ is shown in Figure 9. Test case and test procedure generation diverge from requirements analysis at the Stage 2 translation step. The Stage 1 FOL form is used to generate the inputs needed for test generation.

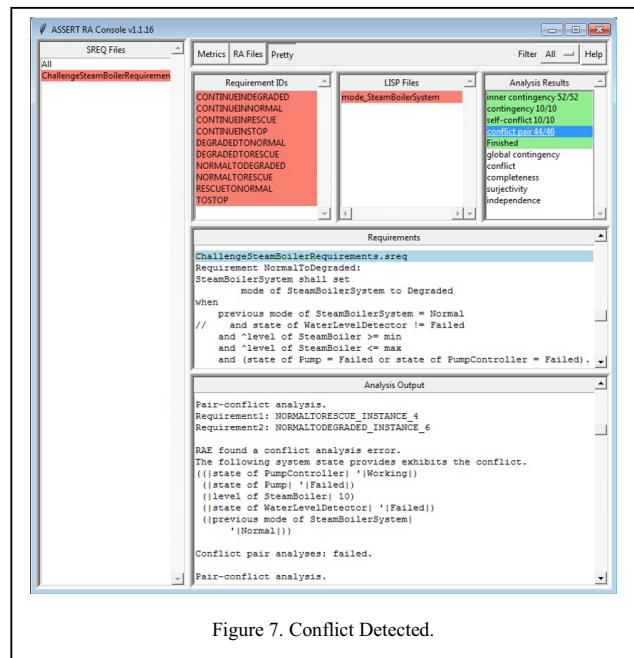


Figure 7. Conflict Detected.

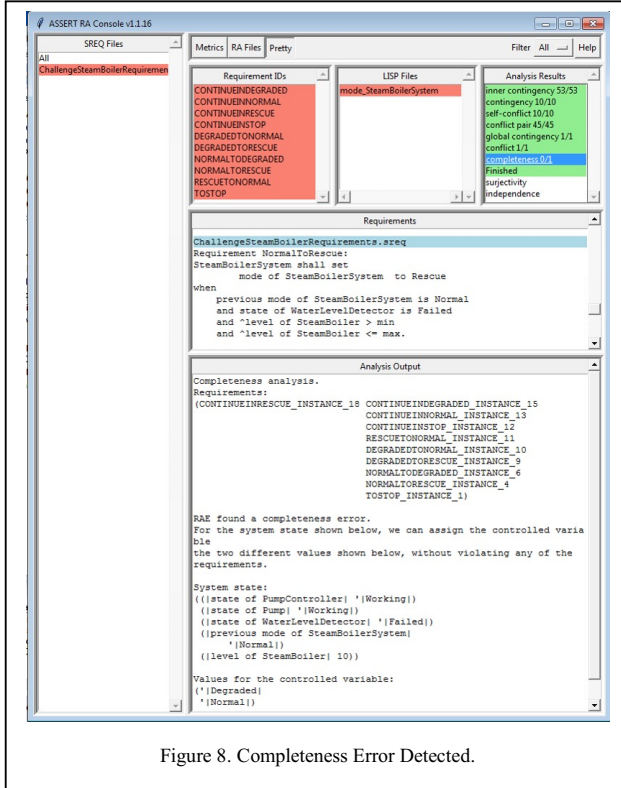


Figure 8. Completeness Error Detected.

The ASSERT™ platform is currently being used on a number of projects within GE. To date these projects have mostly involved functional requirements. In one avionics project, use of the tool for iterative correction and refinement culminated in a set of 1002 requirements with no errors from which 19,276 test cases and 38,272 test procedures were automatically generated. A representative subset of the requirements was identified and the test cases generated were manually verified. Each additional project presents an opportunity to broaden our understanding of what is required to capture requirements that are both human readable and machine analyzable, thus allowing us to improve the functionality and usability of the ASSERT™ platform.

IX. CONCLUSIONS

In this paper we have introduced the ASSERT™ platform and provided an overview of requirements capture and analysis using the tool. ASSERT™ is a work in-progress, benefiting daily from the experiences and feedback from real requirements engineers working to leverage its capabilities to solve real-world problems. We continue to learn what works and what needs work. We are encouraged by what the tool is able to do and see huge opportunities for improvement in the future as we continue to gain experience with different applications, improve its scale and function, and improve the semantics upon which ASSERT is founded.

X. ACKNOWLEDGEMENTS

ASSERT™ development has been a collaborative effort between the GE Global Research Center and GE Aviation Sys-

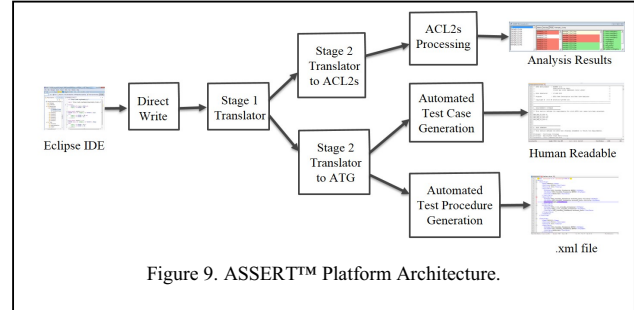


Figure 9. ASSERT™ Platform Architecture.

tems. The authors gratefully acknowledge the significant contributions of Professor Pete Manolios of Northeastern University who has played a key role in designing and implementing the requirements analysis functionality of ASSERT™.

REFERENCES

- [1] R. Feynman, R. Leighton and M. Sands, "The Feynman Lectures on Physics". Volume 2, chapter 25, page 10 (Equation 25.32). [Online] http://www.feynmanlectures.caltech.edu/II_25.html.
- [2] S. Carroll, "The Big Picture: On the Origins of Life, Meaning, and the Universe Itself" (Appendix: The Equation Underlying You and Me), Dutton, 2016.
- [3] E. Borger, E. Riccobene, J. Schmid, "Capturing requirements by abstract state machines: the light control case study," Journal of Universal Computer Science, pp. 597-620, 2000.
- [4] N. Leveson, M. Heimdahl, H. Hildreth, J. Reese, "Requirements specification for process-control systems," IEEE Transaction on Software Engineering, September 1994.
- [5] S. Rayadurgam, A. Joshi, M. Heimdahl, "Using PVS to prove properties of systems modelled in a synchronous dataflow language," International Conference on Formal Engineering Methods, November 2003.
- [6] GitHub SpeAR [Online] <https://github.com/lgwagner/SpeAR>.
- [7] K. Gross, A. Fifarek, J. Hoffman, "Incremental formal methods based design approach demonstrated on a coupled tanks control system," IEEE 17th International Symposium on High Assurance Systems Engineering, 2016.
- [8] J. C. Bauer, "Specification for a software program for a boiler water content monitor and control system", Institute of Risk Research, University of Waterloo, 1993.
- [9] J.-R. Abrial, E. Borger, and H. Langmaack (editors), Formal Methods for Industrial Applications: Specifying and Programming the Steam Boiler Control, LNCS, Vol. 1165, Springer, 1996.
- [10] P. N. Johnson-Laird, "Mental Models: Towards a Cognitive Science of Language, Inference, and Consciousness", Harvard University Press, Cambridge, MA, 1983.
- [11] P. N. Johnson-Laird, "Mental Models: The Computer and the Mind", Harvard University Press, Cambridge, MA, 1988.
- [12] P. N. Johnson-Laird, "Human and Machine Thinking", Lawrence Erlbaum Associates, Hisdale, NJ, 1993.
- [13] OWL Web Ontology Language Reference: W3C Recommendation 10 February 2004, available on-line at <http://www.w3.org/TR/owl-ref/>.
- [14] W3C [Online] <https://www.w3.org/>.

- [15] Semantic Application Design Language (SADL). [Online] <http://sadl.sourceforge.net/index.html>.
- [16] A. Crapo, A. Moitra. Toward a unified English-like representation of semantic models, data, and graph patterns for subject matter experts. *International Journal of Semantic Computing*, Vol. 7, No. 3, 2013, pp. 215-236.
- [17] "Xtext: Language Engineering for Everyone". [Online] <http://www.eclipse.org/Xtext/>.
- [18] "Safety Critical Software and Systems Research @ General Electric", Safe and Secure Systems and Software Symposium (S5), July 12-14, 2016. [Online] http://www.mys5.org/Proceedings/2016/Day_2/2016-S5-Day2_0845_Durling.pdf.
- [19] A. Mavin, P. Wilkinson, S. Gregory, and E. Uusitalo, "Listens Learned (8 Lessons Learned Applying EARS)", IEEE Requirements Engineering Conference (RE16), [Online] https://www.researchgate.net/profile/Alistair_Mavin/publication/308970788_Listens_Learned_8_Lessons_Learned_Applying_EARS/links/57fbb3ca08ae6ce92eb2acd1/Listens-Learned-8-Lessons-Learned-Applying-EARS.pdf
- [20] "SWI-Prolog for the (semantic) web", [Online] <http://www.swi-prolog.org/web/>.
- [21] ACL2s: "The ACL2 Sedan" for Eclipse. [Online] <http://acl2.ccs.neu.edu/acl2s>.
- [22] P. Manolios, "Scalable methods for analyzing formalized requirements and localizing errors", U.S. Patent 9639450 B2, May 2, 2017.

APPENDIX A: COMPLETE SET OF REQUIREMENTS FOR STEAM-BOILER PROBLEM

```
uri "http://sabl.org/ChallengeSteamBoilerRequirements.sreq"
alias reqs.
import "http://sabl.org/SteamBoiler.sabl".
```

Context:

```
SteamBoilerSystem with component SteamBoiler and
  min is 10.0 and // minimum of operational water level
  max is 65.5.    // maximum of operational water level
```

Requirement ToStop:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Stopped
when
  ^level of SteamBoiler < min
  or ^level of SteamBoiler > max.
```

Requirement NormalToRescue:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Rescue
when
  previous mode of SteamBoilerSystem is Normal
  and state of WaterLevelDetector is Failed
  and ^level of SteamBoiler > min
  and ^level of SteamBoiler <= max.
```

Requirement NormalToDegraded:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Degraded
when
  previous mode of SteamBoilerSystem = Normal
  and state of WaterLevelDetector != Failed
  and ^level of SteamBoiler >= min
  and ^level of SteamBoiler <= max
  and (state of Pump = Failed
  or state of PumpController = Failed).
```

Requirement DegradedToRescue:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Rescue
when
  previous mode of SteamBoilerSystem = Degraded
  and state of WaterLevelDetector = Failed
  and ^level of SteamBoiler >= min
  and ^level of SteamBoiler <= max.
```

Requirement DegradedToNormal:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Normal
when
  previous mode of SteamBoilerSystem = Degraded
  and state of WaterLevelDetector != Failed
  and state of Pump != Failed
```

```
and state of PumpController != Failed
and ^level of SteamBoiler >= min
and ^level of SteamBoiler <= max.
```

Requirement RescueToNormal:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Normal
when
  previous mode of SteamBoilerSystem = Rescue
  and state of WaterLevelDetector = Working
  and ^level of SteamBoiler >= min
  and ^level of SteamBoiler <= max.
```

////////// Continue in Existing State //////////

Requirement ContinueInStop:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Stopped
when
  previous mode of SteamBoilerSystem = Stopped.
```

Requirement ContinueInNormal:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Normal
when
  previous mode of SteamBoilerSystem = Normal
  and state of WaterLevelDetector != Failed
  and ^level of SteamBoiler >= min
  and ^level of SteamBoiler <= max
  and state of Pump != Failed
  and state of PumpController != Failed.
```

Requirement ContinueInDegraded:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Degraded
when
  previous mode of SteamBoilerSystem = Degraded
  and state of WaterLevelDetector != Failed
  and (state of Pump = Failed
  or state of PumpController = Failed)
  and ^level of SteamBoiler >= min
  and ^level of SteamBoiler <= max.
```

Requirement ContinueInRescue:

```
SteamBoilerSystem shall set
  mode of SteamBoilerSystem to Rescue
when
  previous mode of SteamBoilerSystem = Rescue
  and state of WaterLevelDetector = Failed
  and ^level of SteamBoiler >= min
  and ^level of SteamBoiler <= max
```